CAMPUS DE EXCELENCIA INTERNACIONAL

Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Metodologías de Desarrollo Web

Master en Ingeniería Web
E.T.S. de Ingeniería en Sistemas Informáticos
Universidad Politécnica de Madrid
Curso 2025-2026



Jennifer Pérez y Agustín Yagüe
Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos

CAMPUS DE EXCELENCIA INTERNACIONAL

Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas Informáticos

Ingeniería del Software e Ingeniería Web



Ingeniería del Software: Definición

QUICK LOOK

What is it? Software engineering encompasses a process, a collection of methods (practice) and an array of tools that allow professionals to build high-quality computer software.

Who does it? Software engineers apply the software engineering process.

Why is it important? Software engineering is important because it enables us to build complex systems in a timely manner and with high quality. It imposes discipline to work that can become quite chaotic, but it also allows the people who build computer software to adapt their approach in a manner that best suits their needs.

What are the steps? You build computer software like you build any successful product, by applying an agile, adaptable process that leads to a high-quality result that meets the needs of the people who will use the product. You apply a software engineering approach.

What is the work product? From the point of view of a software engineer, the work product is the set of programs, content (data), and other work products that are computer software. But from the user's viewpoint, the work product is the resultant information that somehow makes the user's world better.

***Software Engineering: A Practitioner's Approach 8th Edition**, Roger Pressman, Bruce Maxim, 2014.*

Ingeniería del Software: Definición

Question	Answer
What is software engineering?	Software engineering is an engineering discipline that is concerned with all aspects of software production.
What are the fundamental software engineering activities?	Software specification, software development, software validation and software evolution.
What are the key challenges facing software engineering?	Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.
What are the costs of software engineering?	Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
What are the best software engineering techniques and methods?	While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. You can't, therefore, say that one method is better than another.

***Software Engineering, 10th Edition** Ian Sommerville, University of St Andrews, Scotland, ©2016*

Niveles de la Ingeniería del Software



Software Engineering: A Practitioner's Approach 8th Edition, *Roger Pressman, Bruce Maxim, 2014.*

El Pentágono de la Ingeniería del Software



Maciaszek, L.A. and Liong, B.L, Practical Software Engineering. A Case Study Approach, Harlow England, Addison-Wesley, 864p,ISBN 0-321-20465-4, 2005

Ingeniería Web

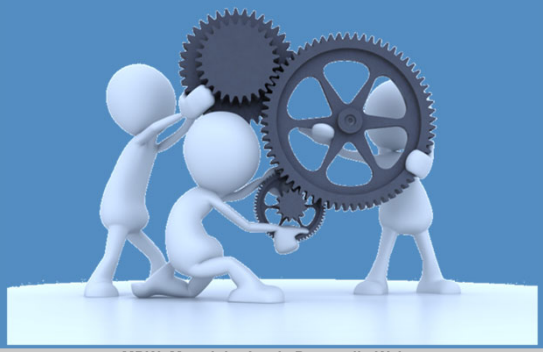


<<***Software engineering*** *is the field of computer science that deals with the building of software systems that are so large or so complex that they are built by a team or teams of engineers>>*

Fundamentals of software engineering 2nd Edition. *Carlo Ghezzi, Mehdi Jazayeri, Dino Mandrioli, Prentice Hall 2003: I-XX, 1-604*

<<***Web engineering*** *es un dominio específico de aplicación de la ingeniería del software en que el proceso de desarrollo se especializa para construir productos software web. Esta especialización hace que emerjan tecnologías especializadas y que aspectos como la interfaces, su modelos de navegación, usabilidad y accesibilidad cobren más relevancia en el proceso de desarrollo del producto que en otros dominios de aplicación. >>*

Proceso SW vs Producto SW



Definición

- ***Producto SW***
  - El ¿QUÉ?
  - El objetivo es desarrollar un producto software
  - **Modelo del Producto Software**
    - Representación abstracta del producto software
- ***Proceso SW***
  - El ¿CÓMO?
  - El objetivo es seguir un proceso que nos especifica los pasos a seguir y tareas a realizar para desarrollar un producto software
  - **Modelo de Proceso Software**
    - Representación abstracta del proceso software

Producto SW

- ***Dominios de Aplicación***
  - Sistemas Software:
    - Conjunto de programas desarrollados para dar soporte a otros programas
  - Aplicaciones Software:
    - Aplicaciones *stand-alone* desarrolladas con una funcionalidad u objetivo de negocio específico.
  - Software Científico o Ingenieril:
    - Sistemas desarrollados para dar un soporte a la ciencia
  - Software Embebido
    - Software alojado en sistemas hardware para dar una función especifica

Producto SW

- ***Dominios de Aplicación***
  - Líneas de Producto Software
    - Software para un sector de mercado (*Marketplace)* y abarcar un gran conjunto potencial de clientes
  - **Aplicaciones Web y Móviles:**
    - **Aplicaciones basadas en navegadores con la capacidad de ser alojadas en dispositivos portables**
  - Software de Inteligencia Artificial:
    - Aplicaciones que utilizan algoritmos para resolver sistema complejos.
    - Ejemplos: Sistemas robóticos, reconocimientos de voz e imagen, juegos, etc.
  - Sistemas legados
    - Programas antiguos que se quedan obsoletos y necesitan adecuarse a nuevas necesidades (funcionales o no funcionales) o al mercado actual

Producto SW

- ***Categorías de Aplicaciones Web***

Category	Examples
Informational	Online newspapers, product catalogs, newsletters, service manuals, online classifieds, online electronic books
Interactive (user-provided information or customized access)	Registration forms, customized information presentation, online games
Transactional	Electronic shopping, ordering goods and services, online banking
Workflow	Online planning and scheduling systems, inventory management, status monitoring
Collaborative work environments	Distributed authoring systems, collaborative design tools
Online communities, marketplaces	Chat groups, recommender systems that recommend products or services, online marketplaces, online auctions
Web portals	Electronic shopping malls, online intermediaries

A. Ginige and S. Murugesan, "Web engineering: an introduction," in IEEE MultiMedia, vol. 8, no. 1, pp. 14-18, Jan.-March 2001, doi: 10.1109/93.923949 , https://ieeexplore.ieee.org/document/923949

Producto SW

- ***Problemas de los Sistemas Web***

Problems of Web-Based Systems Development: A Diagnosis

Most Web developers pay little attention to requirements elicitation and analysis, development methodologies and process, quality, performance evaluation, configuration and project management, and maintainability and scalability. Furthermore, application development heavily relies on the knowledge and experience of individual (or a small group of) developers and their individual development practices rather than standard practices. These systems also lack proper testing and documentation.

The Web's legacy as an information medium rather than an application medium is another problem. Many consider Web development primarily an authoring problem rather than an application development problem. They often get carried away by the myth that "Web development is an art" and it deals with "media manipulation and presentation." Sure, like the process of designing and constructing buildings, Web development has an important artistic side. However, Web developers need to follow some discipline, as in science and engineering.

The complexity of Web-based applications has also grown significantly—from information dissemination (consisting of simple text and images to image maps, forms, Common Gateway Interface, applets, scripts, and style sheets) to online transactions, enterprise-wide planning and scheduling systems, Web-based collaborative work environments, and so on. The complexity of many of these Web-based systems is often deceptive.

Several attributes of quality Web-based systems such as ease of navigation, accessibility, scalability, maintainability, usability, compatibility and interoperability, and security and reliability aren't given the due consideration they deserve during development. Many Web applications also fail to address cultural, privacy, moral, and legal aspects.

Web-based systems development isn't a one-time event, as practiced by many; it's a process with a long life cycle.

Claves del Buen Desarrollo Web:

- Proceso Ingenieril
- Ingeniería de Requisitos
- Análisis
- Testing
- Calidad y Atributos de Calidad
- Evaluación y Aprendizaje

A. Ginige and S. Murugesan, "Web engineering: an introduction," in IEEE MultiMedia, vol. 8, no. 1, pp. 14-18, Jan.-March 2001, doi: 10.1109/93.923949 , https://ieeexplore.ieee.org/document/923949

Producto SW

- ***Características de los Sistemas Web***

Advanced Web-Based Systems
Complex Web pages
Information is dynamic—changes with time and users' needs
Difficult to navigate and find information
Integrated with database and other planning, scheduling, and tracking systems
Requires high performance and continuous availability
Requires a large development team with expertise in diverse areas
Deployed in mission-critical applications

A. Ginige and S. Murugesan, "Web engineering: an introduction," in IEEE MultiMedia, vol. 8, no. 1, pp. 14-18, Jan.-March 2001, doi: 10.1109/93.923949 , https://ieeexplore.ieee.org/document/923949

Producto SW



- **La calidad del proceso afecta a la calidad del producto**
- El software se construye para satisfacer:
  - Un objetivo funcional determinado
  - Un conjunto de atributos y cualidades
    - Determinan la calidad del producto



- El proceso software también debe satisfacer dichas cualidades →Proceso de calidad



Producto SW

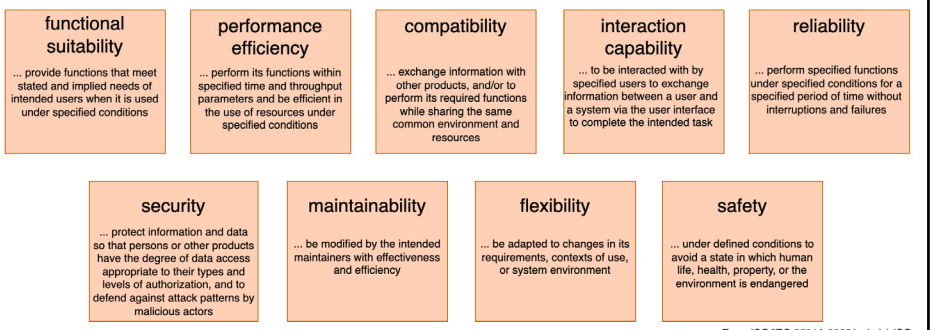
- ***Atributos de Calidad***
  - Característica de un sistema que es posible identificar y evaluar de forma cuantitativa o cualitativa para determinar en qué medida el sistema cumple las expectativas y requisitos de los *stakeholders*.
  - Sirven como *benchmark* para evaluar aspectos críticos del sistema, proporcionando una comprensión integral de la calidad general del sistema y su idoneidad para el fin previsto.
  - Se clasifican en tres categorías:

SYSTEM	BUSINESS	DEVELOPMENT
▪ Availability ▪ Interoperability ▪ Performance ▪ Security ▪ Usability	• Marketability • Cost and benefits • Projected life time • Integration with legacy • Roll back schedule	• Modifiability • Testability • Portability • Maintainability • Reusability



Producto SW

- ***Atributos de Calidad***
  - **ISO/IEC 25010: 2023 SQuaRE (System and Software Quality Requirements and Evaluation)**

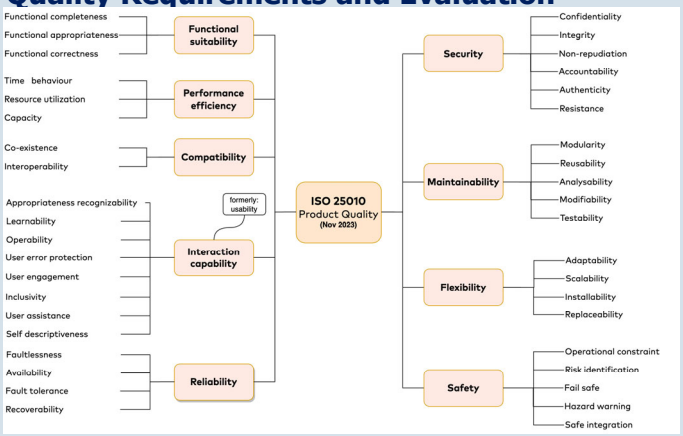


From ISO/IEC 25010:2023(en), (c) ISO.org

https://www.iso.org/obp/ui/#iso:std:iso-iec:25010:ed-2:v1:en

Producto SW

- ***Atributos de Calidad***
  - **ISO/IEC 25010: 2023 SQuaRE (System and Software Quality Requirements and Evaluation**



https://quality.arc42.org/articles/iso-25010-update-2023

Producto SW

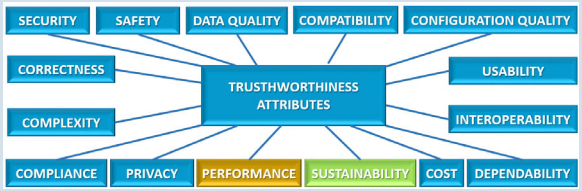
- ***Atributos de Calidad***
  - **ISO/IEC 25010: 2023 SQuaRE (System and Software Quality Requirements and Evaluation**
    - **Quality IN USE Model:**

Grado en que un producto puede **ser utilizado por usuarios** específicos para satisfacer sus necesidades y alcanzar objetivos específicos con eficacia, eficiencia, sin riesgos y con satisfacción en contextos de uso específicos.

Effectiveness
Efficiency
Satisfaction
Usefulness
Trust
Pleasure
Comfort
Freedom from risk
Economic risk mitigation
Health and safety risk mitigation
Environmental risk mitigation
Context coverage
Context completeness
Flexibility

Producto SW

- ***Atributos de Calidad***
  - **ISO/IEC 25010: 2023 SQuaRE (System and Software Quality Requirements and Evaluation**
    - **Trust and Trustworthiness: Esenciales en la INDUSTRIA 4.0**
      - **Trust** es la percepción individual y subjetiva de cada persona, es decir, el usuario o cliente potencial de un producto de software. Representa su creencia en un sistema de software en términos de si va a funcionar correctamente según sus necesidades e intereses individuales.
      - **Trustworthiness** es una cualidad del propio sistema de software y se garantiza mejorando aquellos atributos de calidad que se reconocen como **trustworthiness quality attributes**



Juan Sebastián Ochoa, Vanessa Rodríguez-Horcajo, Jennifer Pérez, Juan Garbajosa, Norberto Cañas, Javier García, Daniel Guamán, ***Sustainability and Performance Trustworthiness*** *of IoT Monitoring Software Architectures in the Edge,* ***Journal of Systems and Software, 2026****, 112801, ISSN 0164-1212, https://doi.org/10.1016/j.jss.2026.112801*

Producto SW

- ***Atributos de Calidad***
  - **ISO/IEC 25010: 2023 SQuaRE (System and Software Quality Requirements and Evaluation**
    - **Quality IN USE Model:**
      - **IN WEB DEVELOPMENT**

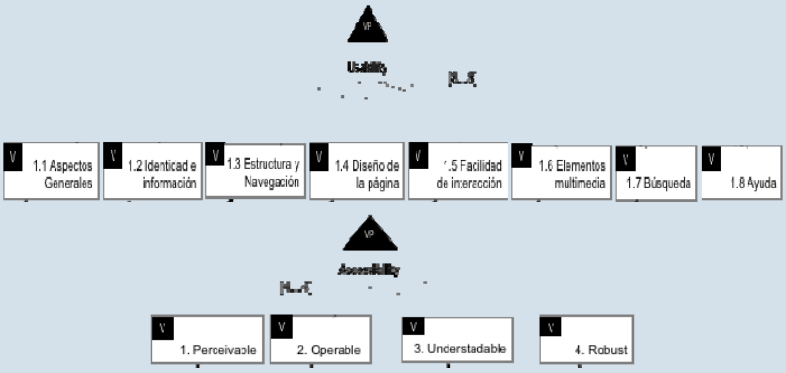
Effectiveness
Efficiency
Satisfaction
Usefulness
Trust
Pleasure
Comfort
Freedom from risk
Economic risk mitigation
Health and safety risk mitigation
Environmental risk mitigation
Context coverage
Context completeness
Flexibility

→ Ease of Use; Experience; Perceived Interaction Quality; Perceived Transaction Quality; Sense of Community

↓ Accessibility; Personalization

Producto SW

- ***Atributos de Calidad***
  - **Usabilidad y Accesibilidad en Entornos Web**



Rodríguez Morales, Germania (2019). Framework para construir plataformas de Recursos Educativos Abiertos (OCW y MOOC) orientadas a la Accesibilidad y Usabilidad. Tesis (Doctoral), E.T.S.I. de Sistemas Informáticos (UPM). https://doi.org/10.20868/UPM.thesis.57249.

Germania Rodríguez, Jennifer Pérez, Samanta Cueva, Rommel Torres, A framework for improving web accessibility and usability of Open Course Ware sites, Computers & Education, Volume 109, 2017, Pages 197-215, ISSN 0360-1315, https://doi.org/10.1016/j.compedu.2017.02.013 .

Producto SW

- ***Atributos de Calidad***
  - **Web content accessibility guidelines (WCAG).**
    - Estándar Internacional de las Pautas de Accesibilidad para el Contenido Web



Free Web Accessibility Checkers WCAG 2.2 : https://achecker.ca/, https://www.tawdis.net/#

https://www.w3.org/WAI/standards-guidelines/wcag/es

Producto SW

- ***Atributos de Calidad***
  - Estándar Internacional ISO 9241-11:2018 **Ergonomics of human-system interaction** Part 11: **Usability**

Atributo ISO 9142-11	Aspectos	Criterio
EFICIENCIA	Evaluación Automática	Evaluación Automática
EFICACIA		
SATISFACCIÓN	ASPECTOS GENERALES	AG1: Objetivos del sitio web concretos y bien definidos
		AG2: Contenidos y servicios ofrecidos precisos y completos
		AG3: Estructura general del sitio web orientada al usuario
		AG6: Diseño general del sitio web coherente
		AG7: Se utiliza el idioma del usuario
	IDENTIDAD E INFORMACIÓN	II7: Se ofrece información sobre el autor, fuentes y fechas de creación y revisión del documento en artículos, noticias, informes
	ESTRUCTURA Y NAVEGACIÓN	EN2: Estructura de organización y navegación adecuadas
		EN4: Control del número de elementos y de términos por elemento en los menús de navegación
		EN5: Equilibrio entre profundidad y anchura en el caso de estructura jerárquica
		EN6: Enlaces fácilmente reconocibles como tales
		EN7: La caracterización de los enlaces indica sus estados (visitados, activos)
		EN8: Se ha evitado la redundancia de enlaces
		EN9: Se ha evitado la existencia de enlaces rotos
		EN13: Hay elementos de navegación que orienten al usuario acerca de donde está y cómo deshacer su navegación (migas, pestañas coloreadas)
	LAYOUT DE LA PÁGINA	LA8: La versión impresa de la página es correcta.
		LA9: El texto de la página se lee sin dificultad.
	ENTENDIBILIDAD Y FACILIDAD EN LA INTERACCIÓN	EF1: Se emplea un lenguaje claro y conciso.

ISO 9241-11:2018 - Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts, https://www.iso.org/standard/63500.html

Producto SW

- ***Reglas Básicas de la Construcción del Software***
- Utilizar un proceso de desarrollo manejable y conocido.
- Tener en cuenta la dependencia y el rendimiento
- Es vital conocer y gestionar los requisitos del sistema
- Reutilizar código siempre que sea adecuado y se pueda



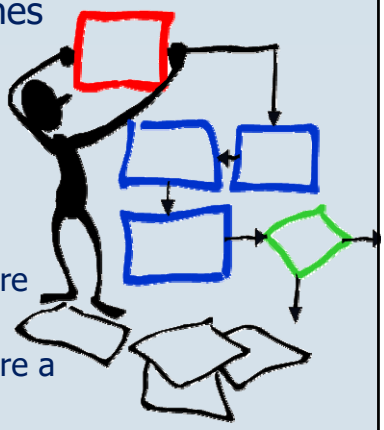
Producto SW

- ***Principios de la Construcción del Software***
- Minimizar la complejidad
  - Modularidad
  - Separación de asuntos
  - Cohesión y Acoplamiento
  - Abstracción
  - Generalización
- Anticipación al cambio
  - Desarrollo incremental
- Construcción para Verificar
  - Rigor y Formalidad
- Estándares en Construcción



Proceso

- El ***proceso software*** es un conjunto de actividades, acciones o tareas para construir un producto.
  - Sus resultados constituyen el producto software.
  - Puede consistir en:
    - Desarrollar el producto software desde cero
    - Desarrollar el producto software a partir de sistemas existentes mediante su extensión o modificación



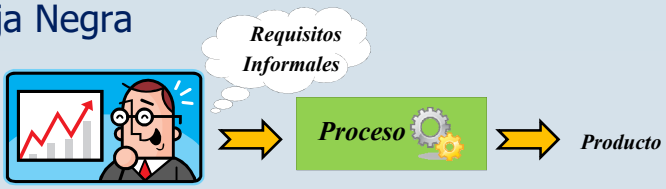
Proceso

- La implementación de software es el proceso de convertir la especificación de un sistema en un sistema ejecutable.
- La especificación de un proceso software en ingeniería del software es personalizable, no es rígida

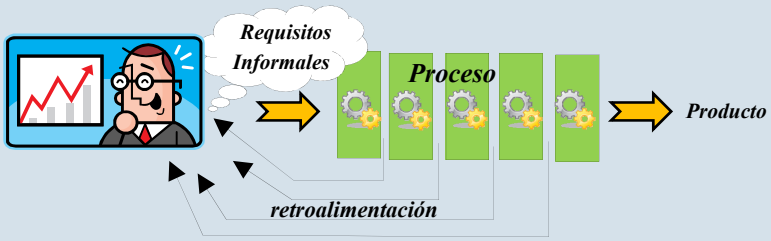


Proceso

- *Vistas*
- Caja Negra

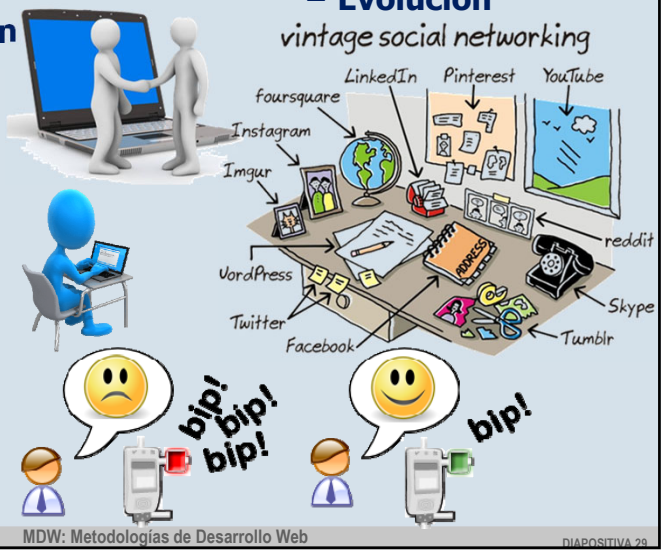


- Caja Blanca

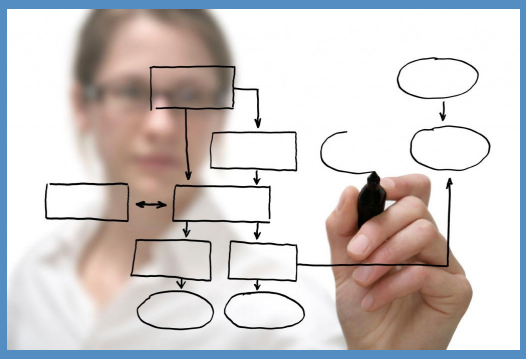


Proceso

- *Actividades Elementales*
  - **Especificación**
    - Análisis
    - Definición
  - **Desarrollo**
    - Diseño
    - Programación
  - **Validación**
  - **Evolución**



Modelos de Proceso Software



Paradigmas de Desarrollo Software

– Definición

> ***<<Conjunto de teorías, estándares y métodos que juntos representan una forma de organizar el conocimiento>>***
> *Thomas Kuhn,* ***The Structure of Scientific Revolutions****, 2nd edition, Chicago: University of Chicago Press, 1970*

- Establece la conceptualización de los activos que llegan a constituir el programa final

– Paradigmas:

- Paradigma declarativo
- Paradigma funcional (procedural, imperativo, estructurado)
- Paradigma orientado a objetos
- Paradigma basado en componentes
- Paradigma orientado a aspectos
- Etc.

Paradigmas de Desarrollo Software

- ***Paradigma Declarativo***

Describen el problema QUÉ se quiere solucionar

- **Lenguajes de programación**
  - Los lenguajes lógicos, como Prolog.
  - Los lenguajes algebraicos, como Maude y SQL.
  - Los lenguajes funcionales, como Haskell y Erlang. OBJ, Axis.

Paradigmas de Desarrollo Software

- ***Paradigma Declarativo***

```
padrede('juan', 'maria'). % juan es padre de maria
padrede('pablo', 'juan'). %pablo es padre de juan
padrede('pablo', 'marcela'). % pablo es padre de marcela
padrede('carlos', 'debora'). % carlos es padre de debora

% A es hijo de B si B es padre de A
hijode(A,B) :- padrede(B,A).
% A es abuelo de B si A es padre de C y C es padre B
abuelode(A,B) :-
   padrede(A,C),
   padrede(C,B).
% A y B son hermanos si el padre de A es también el padre de B y si A y B no son lo mismo
hermanode(A,B) :-
   padrede(C,A) ,
   padrede(C,B),
   A \== B.

% A y B son familiares si A es padre de B o A es hijo de B o A es hermano de B
familiarde(A,B) :-
   padrede(A,B).
familiarde(A,B) :-
   hijode(A,B).
familiarde(A,B) :-
   hermanode(A,B).
```

Paradigmas de Desarrollo Software

- ***Paradigma Imperativo***

Describen CÓMO solucionar un problema

- **SUBTIPOS**
  - Funcional: Pascal, COBOL, etc.
  - OO: Java, C#, etc.
  - O-Aspectos: Aspect, Spring AOP, etc.

Paradigmas de Desarrollo Software

- ***Paradigma Funcional***

```
{
  Author:      John Citizen
  Date:        April 3, 2011
  Student      ID: 1234567
  FileName:    TestSwap
  Description: Swaps two inputed user values around
}

program TestSwap;

{
  Procedure:   Swap
  Input:       v1, v2 : Integers
  Output:      v1, v2 : Integers
  Description:Swaps the values of passed in Integers
}
procedure Swap(var v1, v2 : Integer);
var temp : Integer;
begin
  temp:=v1;            {sets temp to value v1}
  v1:=v2;              {sets the value of v1 to v2}
  v2:=temp;            {sets the value or v2 to temp}
end;

procedure Main();
var
  v1,v2 : Integer;
begin
  Write('Enter a number '  );
  ReadLn(v1);                                  {Read user input a
  Write('Enter another number '  );
  ReadLn(v2);                                  {Read user input a
  WriteLn ('v1 is :',v1,' and v2 is :',v2);
  Swap(v1,v2);                                 {Calls Swap Proced
  WriteLn ('v1 is now :',v1,' and v2 is now :',v2);
end;

begin
  Main();
end.
```

MDW: Metodol

```
Editor - [D:\...\PrimerCobol\HOLA.COB]
File  Edit  Locate  View  Options  Window  Help
000010 IDENTIFICATION DIVISION.
000020 PROGRAM-ID.    HOLA.
000030 ENVIRONMENT    DIVISION.
000040 DATA           DIVISION.
000050 WORKING-STORAGE SECTION.
000130 77  VARIABLE-A  PIC 9(3) VALUE ZERO.
000131 77  VARIABLE-B  PIC 9(3) VALUE ZERO.
000132 77  SUMA        PIC 9(5) VALUE ZERO.
000133 77  CADENA      PIC X(35) VALUE SPACES.
000134 77  FECHA       PIC X(6).
000135 77  HORA        PIC X(6).
000136 77  DIV         PIC 9(6)V99 VALUE ZERO.
000137 77  NUMERO-1    PIC 9 VALUE 2.
000138 77  NUMERO-2    PIC 9 VALUE 2.
000139 01  REG-BLOQ.
000140     02 TABLA-BLOQUEOS OCCURS 50
000141        ASCENDING KEY B-ORG INDEXED BY I.
000142        03 B-ORG  PIC 9(3).
000143        03 B-BLQ  PIC X.
000144 PROCEDURE DIVISION.
000150 * EMBEDDED EXCEPTION DECLARATION
000151     PERFORM 10 TIMES
000161             DISPLAY "Hola Mundo!!!"
000162     END-PERFORM.
000163     MOVE 1 TO VARIABLE-A.
000164     MOVE 2 TO VARIABLE-B.
000165     COMPUTE SUMA = VARIABLE-A + VARIABLE-B.
000166     DISPLAY "LA SUMA ES:" SUMA.
000167     MOVE "ESTE ES MI PRIMER PROGRAMA COBOL" TO
000168          CADENA.
000169 * EN COBOL/400 DIVIDIR ENTRE CERO NO DA ERROR EN COMPUTE
000170     COMPUTE DIV = NUMERO-1 / NUMERO-2.
000171     DISPLAY CADENA.
000172     ACCEPT FECHA FROM DATE.
000173     ACCEPT HORA  FROM TIME.
000174     DISPLAY "HOY ES " FECHA.
000175     DISPLAY "SON LAS " HORA.
```

Paradigmas de Desarrollo Software

- ***Paradigma OO***
  - Se acerca al mundo real
  - Aumenta la comprensibilidad
  - Es parecido a como se afrontan los problemas
  - Todo son objetos
  - Un objeto es una instancia de una clase
  - Una clase define las propiedades y la funcionalidad de un tipo de objetos
  - Clase coche:



Paradigmas de Desarrollo Software

- ***Paradigma OO***

```
public class Taller extends Object{

     private int id;
     private String nombre;
     private String direccion;
     private int codigoPostal;
     private String ciudad;
     private Vector<EmpleadoAsalariadoTaller> empleadosAsalariados;
     private Vector<EmpleadoPorHorasTaller> empleadosPorHoras;

     public Taller(int id, String nombnre, String direccion, int codigoPostal,
                   String ciudad) {
          this.id = id;
          this.nombre = nombre;
          this.direccion = direccion;
          this.codigoPostal = codigoPostal;
          this.ciudad = ciudad;
          this.empleadosAsalariados = new Vector<EmpleadoAsalariadoTaller>(0,1);
          this.empleadosPorHoras = new Vector<EmpleadoPorHorasTaller>(0,1);
     }

     public void setId(int id) {
          this.id = id;
     }
 ....
```

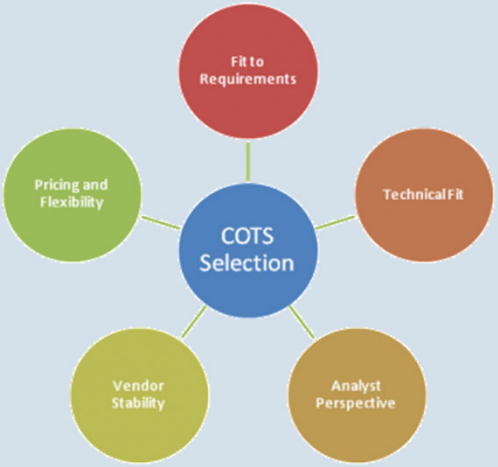
Paradigmas de Desarrollo Software

- ***Paradigma Basado en Componentes***
  - **Desarrollo de Software Basado en Componentes - Component-Based Software Development (CBSD)**
    - Componente
      - Implementación
        - » Un componente es un paquete de código
      - General
        - » Un componente es un artefacto software que ha sido construido para ser reutilizado



Paradigmas de Desarrollo Software

- ***Paradigma Basado en Componentes***
  - **CBSD**
    - Componente
      - **COTS (Commercial Off-The-Self)**
        - » Compra de componentes software a terceros
        - » Cajas negras
        - » Componentes comercializadas para su venta



Varanasi, ***COTS Vendor Selection Criteria,*** *2010*

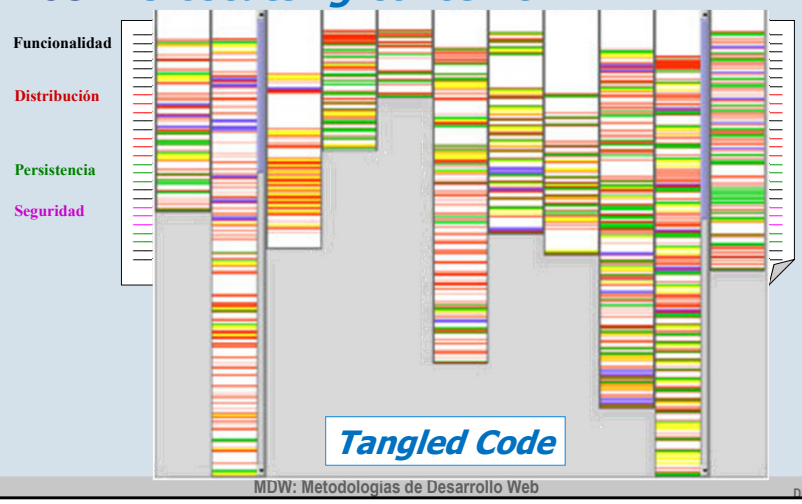
Paradigmas de Desarrollo Software

- ***Paradigma Orientado a Aspectos***
  - **Desarrollo Software Orientado a Aspectos – Aspect-Oriented Software Development (AOSD)**
    - Extensión de ***Aspect-Oriented Programming (AOP)*** a todas las etapas del ciclo de vida software.
    - El origen de AOP es el lenguaje de programación AspectJ
    - AOP ha surgido como una forma innovadora de aplicar la separación de asuntos (***Separation of Concerns (SoC)***) en desarrollo software
      - SoC promueve tratar los distintos concerns de un sistema software de forma individualizada

> ***Concern:*** *<<…those interests which pertain to the system's development, its operation or any other aspects that are critical or otherwise important to one or more stakeholders>>* ***ANSI/IEEE Std 1471-2000***

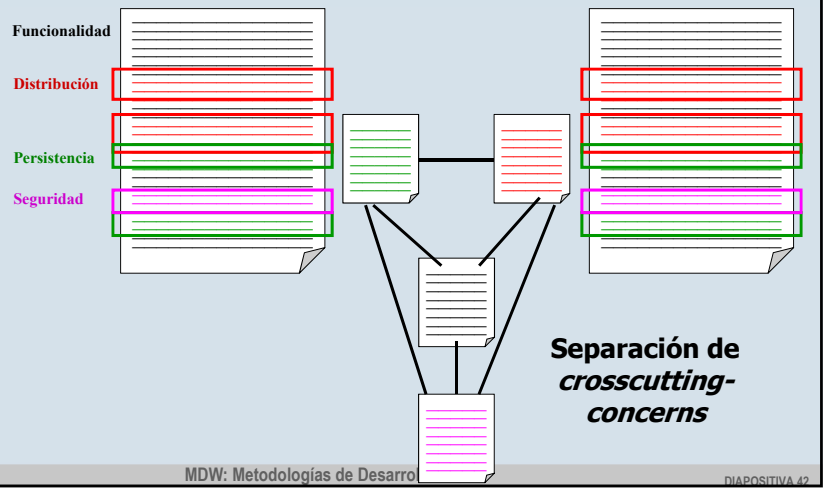
Paradigmas de Desarrollo Software

- ***Paradigma Orientado a Aspectos***
  - **AOSD: *Croscutting-concerns***



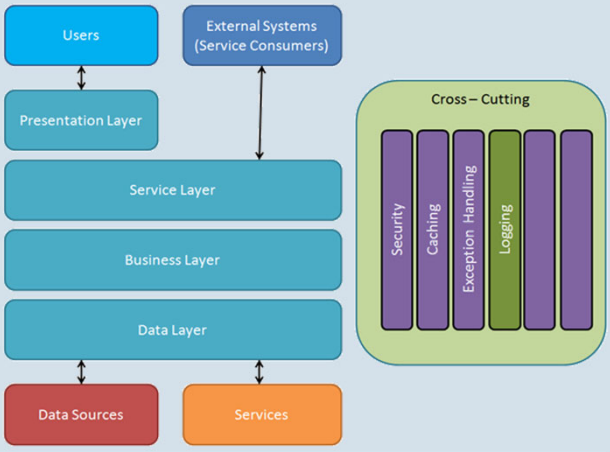
Paradigmas de Desarrollo Software

- *Paradigma Orientado a Aspectos*
  - AOSD: *Croscutting-concerns*



Paradigmas de Desarrollo Software

- *Paradigma Orientado a Aspectos*
  - AOSD: *Croscutting-concerns*



Paradigmas de Desarrollo Software

- *Paradigma Orientado a Aspectos*
  - **AOSD: *AspectJ***

Código Base

```
class Account {
// Variable Definition
   private string ID;          // Account ID
   private float money;        // Money on deposit

//Definition of methods
//Account's constructor
   public Account (string ID, float money)
   {
         this.ID = ID;
         this.money = money;
   }

//This method returns the money that has the
account
   public float Balance ()
   {
     return money;
   }
… … //Other Methods
}
```

Join Point

Join Point

Código Aspecto

```
Aspect AccountTracking{
int ncalls;
pointcut NumberMoneyQuery():
   calls (float Account.balance ());
after NumberMoneyQuery():
{
     System.out.println("The account has
                         been consulted"));
     ncalls++;
   }
}
```

PointCut

Advice

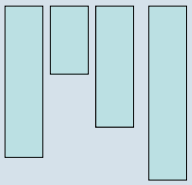
Paradigmas de Desarrollo Software

- *Paradigma Orientado a Aspectos*
  - **AOSD: *Weaving:*** Entrelaza Aspectos con el código base.

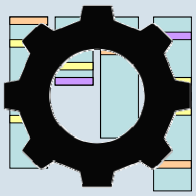
ESTÁTICO

Código Aplicación | **Compilación** | **Ejecución**

Código base



WEAVING



Aspectos





Paradigmas de Desarrollo Software

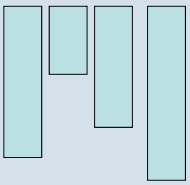
- ***Paradigma Orientado a Aspectos***
  - **AOSD:** ***Weaving:*** Entrelaza Aspectos con el código base.

DINÁMICO



Código Aplicación **Compilación** **Ejecución**

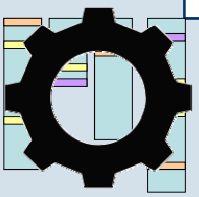
Código base



Aspectos



WEAVING



Paradigmas de Desarrollo Software

- ***Paradigma Orientado a Aspectos***
  - **AOSD:**

Lenguaje	Lenguaje base	Weaving	Propósito
AO4BPEL	BPEL	Dinámico	Específico
ASPECTC	C	Estático	General
ASPECTC++	C++	Estático	General
ASPECTJ	Java	Ambos	General
SPRING AOP	Java	Dinámico	General
ASPECTS	Squeak/Smaltalk	Dinámico	General
CAESARJ	Java	Dinámico	General
CARMA	Java	Estático	General
COMPOSE*	Independiente	Estático	General
D (COOL y RIDL)	Cualquier LOO	Estático	Específico
FUSEJ	Java	Dinámico	Específico
HYPERJ	Java	Dinámico	General
JAC	Independiente	Dinámico	General
JASCO	Java	Ambos	General
JBOSS AOP	Java	Dinámico	General
JPAL	Independiente	Dinámico	General
MALAJ	Java	Dinámico	Específico

Ciclo de Vida vs Modelo de Proceso





- El ***Ciclo de Vida Software*** es la vida de un producto software:
  - Desde el comienzo de una idea, pasando por su desarrollo e implantación hasta que es retirado.
  - El proceso software sigue un modelo
- El ***Modelo de Proceso Software*** intenta organizar el ciclo de vida del software:
  - Definiendo actividades relacionadas con la producción de software
  - Estableciendo un orden entre las actividades
  - Definiendo relaciones entre las actividades
  - El paradigma que utilicemos influye en las tareas a realizar en cada una de las actividades que conforman el modelo de proceso software



Objetivo

*<<determine the **order of stages** involved in software development and evolution, and to establish the **transition criteria** for progressing from one stage to the next. These include completion criteria for the current stage plus choice criteria and entrance criteria for the next stage. Thus a process model **addresses** the following software project **questions**:*

What shall we do next?
How long shall we continue to do it?>>

*Boehm, B. W. "**A Spiral Model of Software Development and Enhancement**", 1988.*

Actividades Fundamentales

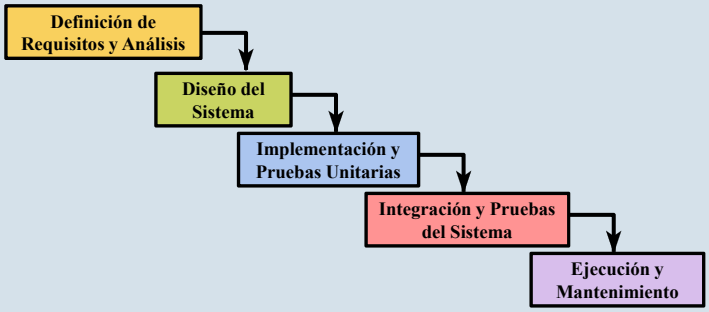
- Existe una gran variedad de modelos de proceso software con actividades diferentes y con distinto orden y prioridad.
- Es posible detectar una serie de actividades fundamentales que todos tienen en común:
  - Especificación de **Requisitos**: Definición de funcionalidad y restricciones.
  - **Diseño e Implementación**: de la funcionalidad y las restricciones definidas en la especificación.
  - **Validación** del software: Validar y Verificar que la funcionalidad y las restricciones diseñadas e implementadas son correctas.
  - Integración e **Implantación**
  - **Mantenimiento** del software y su instalación
  - **Evolución** del software: Existen cambios en las necesidades del software implementado.

Clasificación

- **Modelo en Cascada**
  - Especificación y desarrollo separados y en distintas fases
- **Desarrollo Incremental o Evolutivo**
  - Especificación y desarrollo están intercalados en el tiempo
- **Desarrollo de Sistemas Formales**
  - Un modelo de sistema matemático que se trasforma formalmente en implementación
- **Desarrollo Basado en Reutilización**
  - El sistema se ensambla con componentes que existían previamente
- **Desarrollo Orientado a la Implantación**
  - Modelos que se centran en el proceso de entrega e implantación del producto

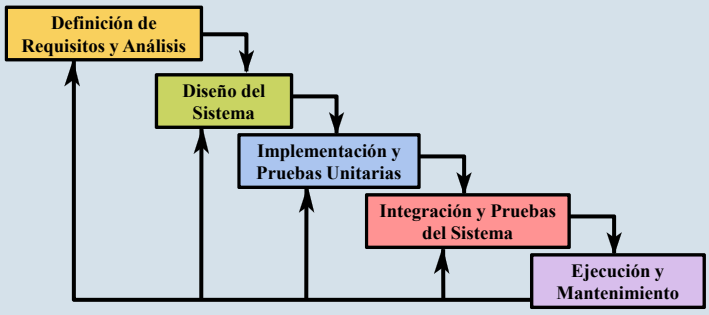
Modelos en Cascada

- Cascada o Clásico
- Cascada o Clásico con Retroalimentación
- Cascada o Clásico con Prototipado



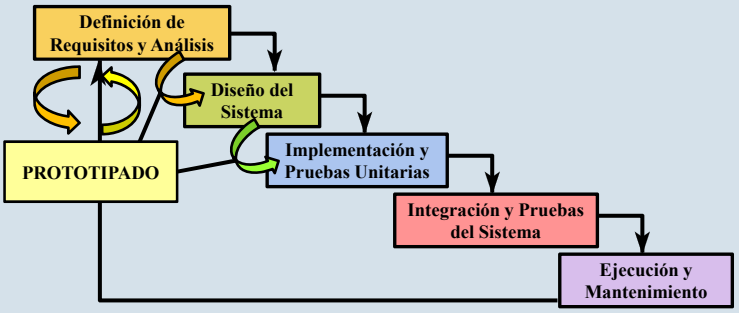
Modelos en Cascada

- Cascada o Clásico
- Cascada o Clásico con Retroalimentación
- Cascada o Clásico con Prototipado



Modelos en Cascada

- Cascada o Clásico
- Cascada o Clásico con Retroalimentación
- Cascada o Clásico con Prototipado



Modelos en Cascada

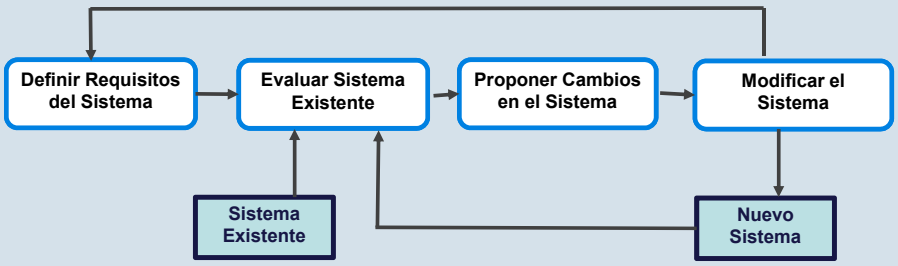
- Cascada o Clásico
- Cascada o Clásico con Retroalimentación
- Cascada o Clásico con Prototipado

	Prototipo Desechable	Prototipo Evolutivo
¿Qué desarrollar?	Las partes problemáticas	La base sólida y bien entendida y consensuada
Objetivo	Minimizar el tiempo Consenso con el Cliente	Fácil Extensibilidad y Modificación Consenso Cliente
Propósito Final	Desecharlo	Incluirlo en el Producto Software Final

Desechable vs Evolutivo

Desarrollo Incremental o Evolutivo

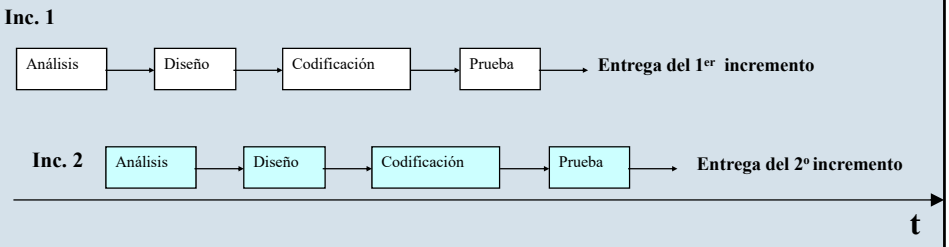
- Modelo Evolutivo
- Modelo Incremental
- Modelo en Espiral
- RUP



Software Engineering: A Practitioner's Approach 8th Edition, *Roger Pressman, Bruce Maxim, 2014.*

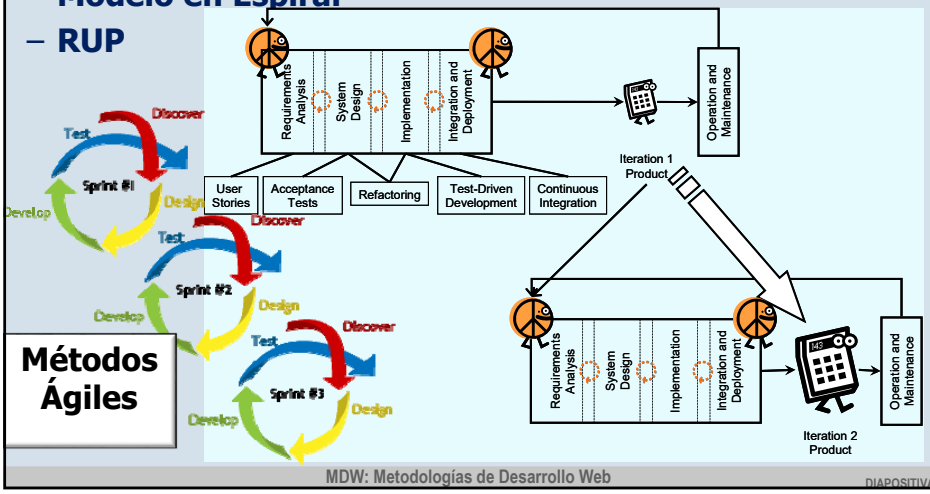
Desarrollo Incremental o Evolutivo

- Modelo Evolutivo
- Modelo Incremental
- Modelo en Espiral
- RUP



Desarrollo Incremental o Evolutivo

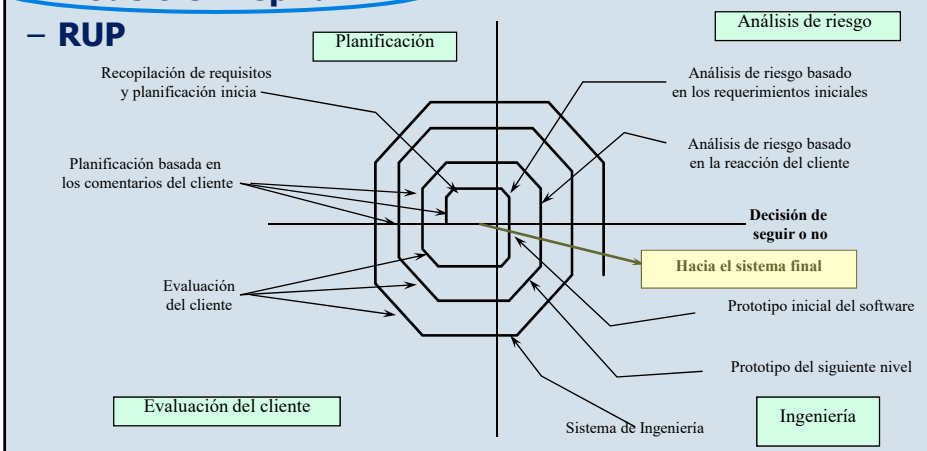
- Modelo Evolutivo
- Modelo Incremental
- Modelo en Espiral
- RUP



Métodos Ágiles

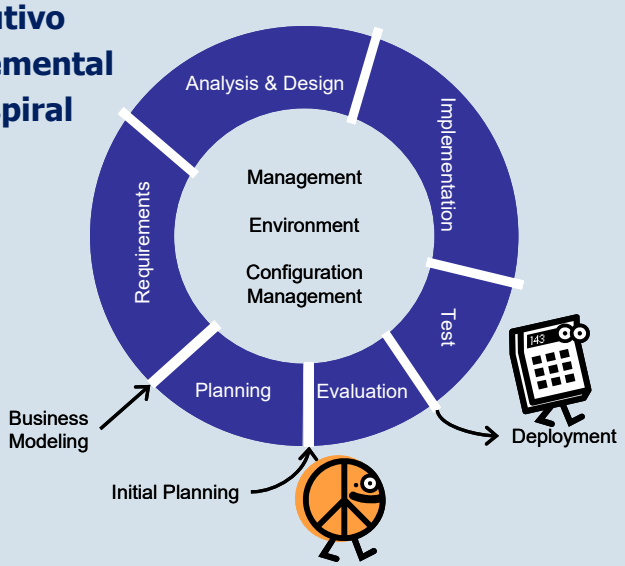
Desarrollo Incremental o Evolutivo

- Modelo Evolutivo
- Modelo Incremental
- Modelo en Espiral
- RUP



Desarrollo Incremental o Evolutivo

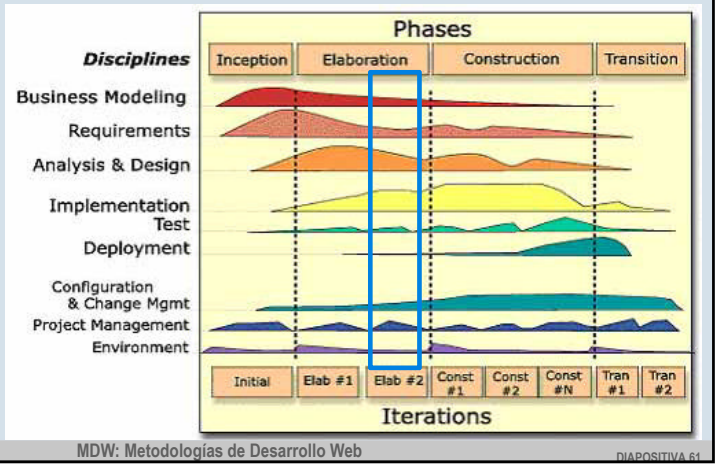
- Modelo Evolutivo
- Modelo Incremental
- Modelo en Espiral
- RUP



Desarrollo Incremental o Evolutivo

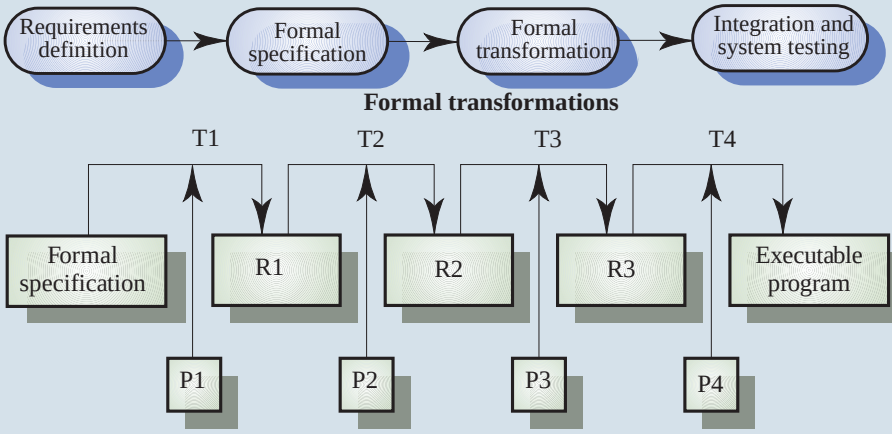
- Modelo Evolutivo
- Modelo Incremental
- Modelo en Espiral
- RUP





Desarrollo de Sistemas Formales

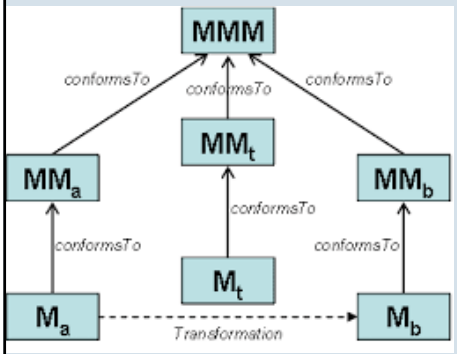
– Modelo General



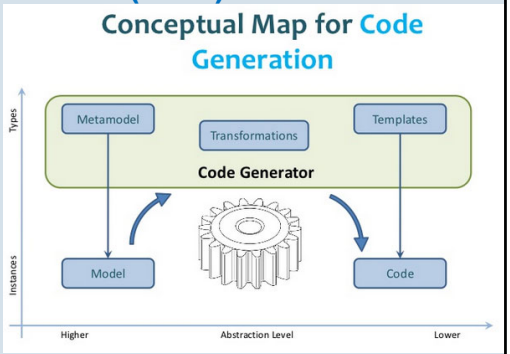
Desarrollo de Sistemas Formales

– Model-Driven Development

Model To Model (M2M), Model To Code (M2C), Model to Text (M2T)



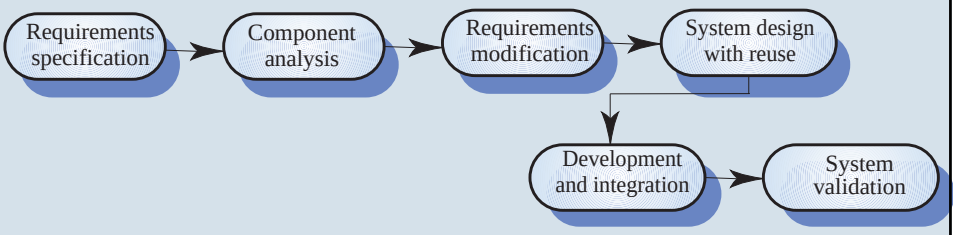
ATL/Concepts, https://wiki.eclipse.org/ATL/Concepts , 2012



Pedro J. Molina, Introducing Model Driven Software Development, Code Generation *2011, Cambridge*

Desarrollo Basado en Reutilización

– Modelo General



Software Engineering: A Practitioner's Approach 8th Edition, *Roger Pressman, Bruce Maxim, 2014.*

Desarrollo Basado en Reutilización

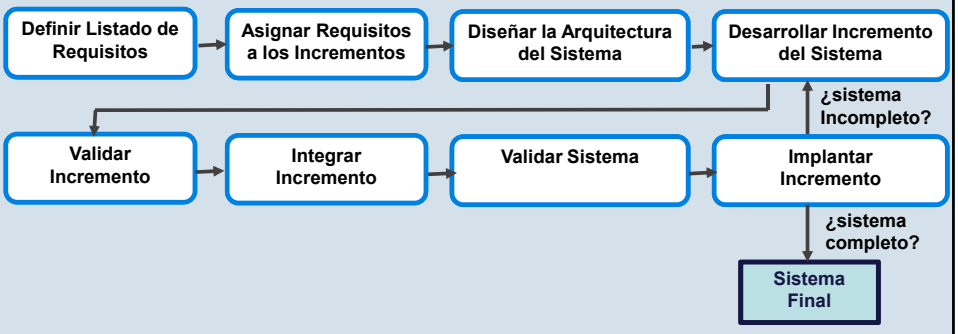
– Modelo General

- **Variantes dependiendo de la unidad de reutilización:**
  - **Componentes**
    - » **COTS (*Commercial Off-The-Shelf*)**
    - » **Componentes SW**
  - **Funciones o Servicios**
  - **Familias:**
    - » **Plataforma**
    - » **Configuración**
    - » **Funcionalidad**

Desarrollo Orientado a la Implantación

– Entrega Incremental

– Despliegue Operacional

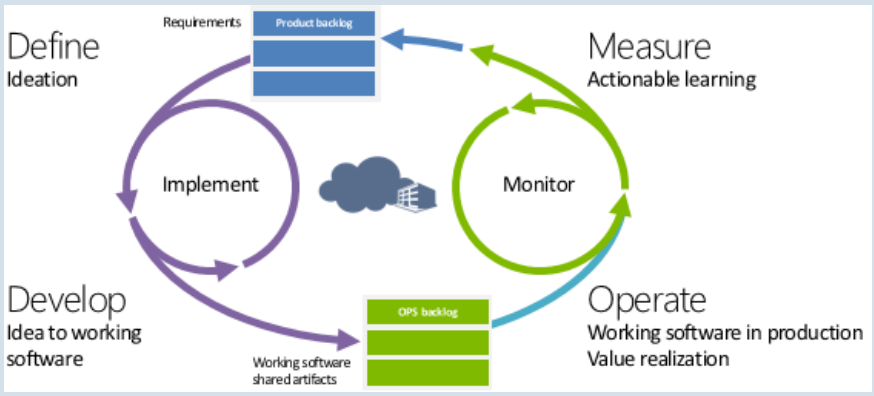


Software Engineering: A Practitioner's Approach 8th Edition, *Roger Pressman, Bruce Maxim, 2014.*

Desarrollo Orientado a la Implantación

– Entrega Incremental

– Despliegue Operacional

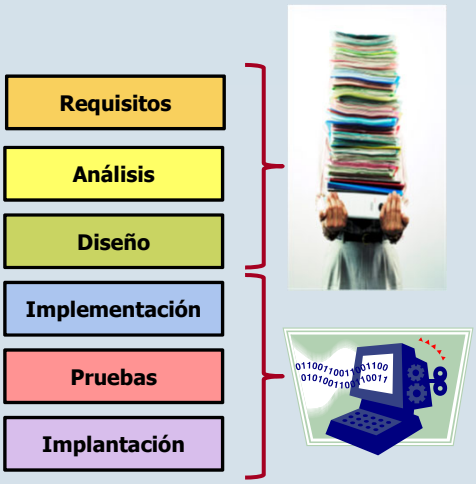


Enfoques de Desarrollo Software



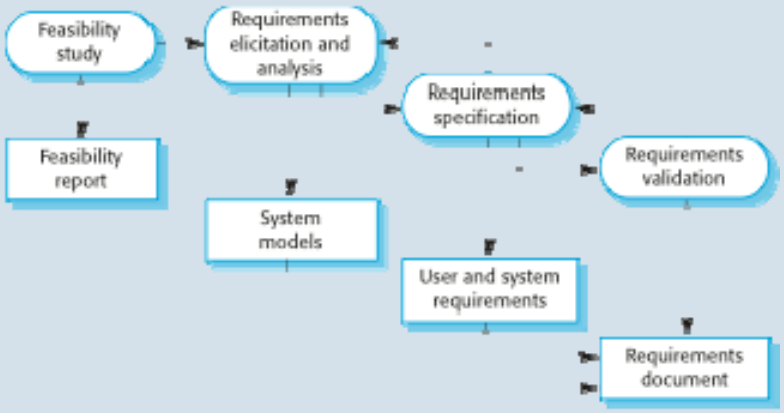
Desarrollo de Software Tradicional

– Marco



Desarrollo de Software Tradicional

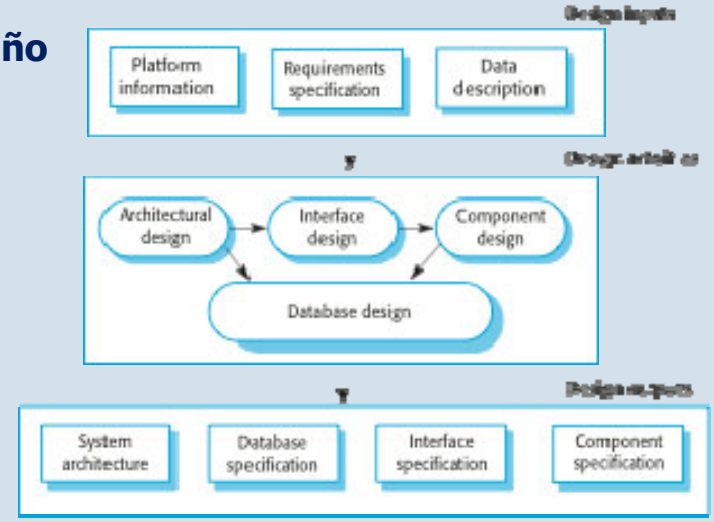
– Ingeniería de Requisitos



Software Engineering: A Practitioner's Approach 8th Edition, *Roger Pressman, Bruce Maxim, 2014.*

Desarrollo de Software Tradicional

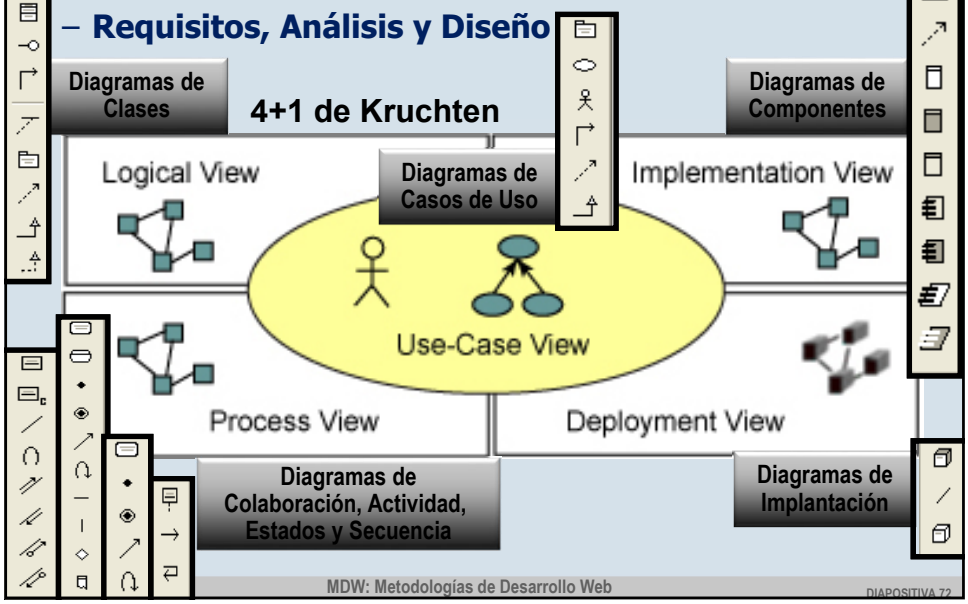
– Diseño



Software Engineering: A Practitioner's Approach 8th Edition, *Roger Pressman, Bruce Maxim, 2014.*

Desarrollo de Software Tradicional

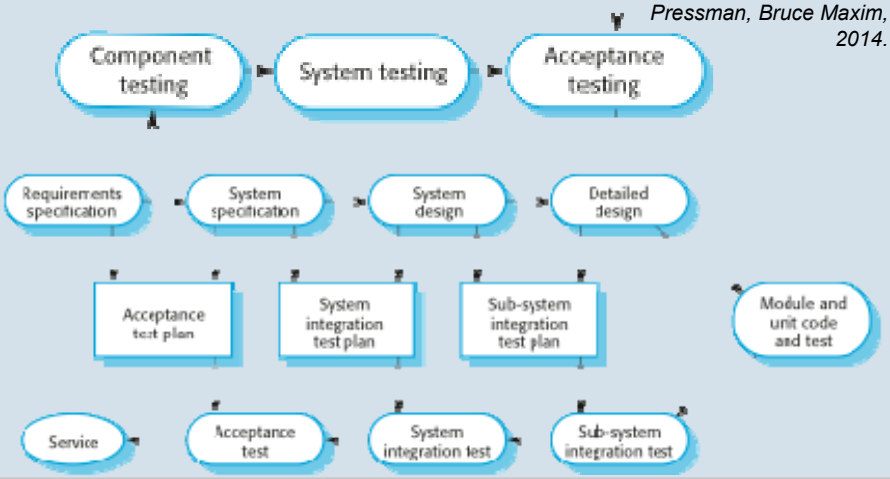
– Requisitos, Análisis y Diseño



Desarrollo de Software Tradicional

– Verificación y Validación

Software Engineering: A Practitioner's Approach 8th Edition, *Roger Pressman, Bruce Maxim, 2014.*



Desarrollo de Software Ágil

AGILE MANIFESTO

Values

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

Principles

Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.

Working software is the primary measure of progress

Continuous attention to technical excellence and good design enhances agility.

Simplicity--the art of maximizing the amount of work not done--is essential.

The best architectures, requirements, and designs emerge from self-organizing teams.

Desarrollo de Software Ágil

– Metologías Ágiles

- Scrum
- Extreme Programming (XP)
- Lean Software Development
- Kanban
- Test-Driven Development(TDD)
- Crystal Methods (Cockburn's Crystal Family Methodologies)
- DSDM (Dynamic Systems Development Method)
- Adaptative Software Development
- Feature-Driven Development
- ....



Desarrollo de Software Ágil

Metodologías Pesadas

- **Desarrollo Predictivo**
  - **Decisiones**:
    - Se toman en las primeras fases del ciclo de vida
    - Se especifican mediante modelos gráficos (UML)
    - Se interpretan y materializan en la fase de desarrollo



Metodologías Ágiles

- **Desarrollo Adaptativo**
  - **Preparado para soportar cambios**
  - **Desarrollo Iterativo**
    - **Feedback**
      - ¿cuál es el estado del sistema?
      - ¿nuevos requisitos?
    - **Pruebas de integración**
      - ¿Errores en el código?
      - ¿Requisitos no satisfechos?



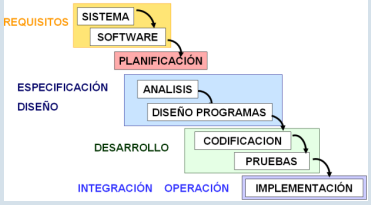
Desarrollo de Software Ágil

Metodologías Pesadas

- **Orientadas a la documentación**:



- **Orientadas al proceso**



Metodologías Ágiles

- **Orientado al código**
  - Programación por parejas



- **Orientadas a las personas**



Desarrollo de Software Ágil

– Ámbito de aplicación: Pesadas vs Ágiles

Metodología pesada	Metodología ágil
Requisitos que cambian ocasionalmente	Requisitos volátiles, inciertos y que cambian frecuentemente
Grupo como roles a desempeñar y número de recursos	Grupo visto como cada una de las personas que lo componen
Existe un contrato con un precio establecido	No existe un contrato fijado (se va adaptando)
El cliente no forma parte del proceso de desarrollo	Cliente es activo en el proceso de desarrollo
Grupos grandes	Grupos no muy grandes (entre 5 y 10 personas)

Desarrollo de Software Automatizado (Desarrollo SW Dirigido por Modelos, MDD)

– Software Factories



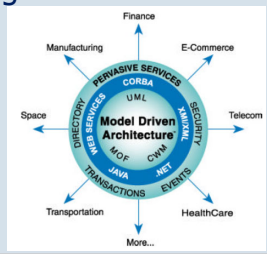
- **Impulsado por Microsoft**
- **Soportado por DSL Tools**
- **Modelado Especifico de Dominio para la producción de software**
  - **Personalización masiva**
  - **Generación de código**
- **Se basa en la mejora de calidad y desarrollo software mediante la reutilización de arquitecturas, componentes software, técnicas y herramientas.**

Desarrollo de Software Automatizado (Desarrollo SW Dirigido por Modelos, MDD)

– Model-Driven Architecture

- Propuesto por OMG
- Soportado por herramientas de gestión de modelos libres
  - Por ejemplo: EMF, GMF, Medini QVT, etc
- Enfoque de desarrollo que tiene como objetivo resolver el problema del coste del cambio de tecnología o integración en distintas plataformas





Desarrollo de Software Basado en Líneas de Producto Software

– *Software Product-Line Engineering (SPLE)*

- Fundamentadas en:





Desarrollo de Software Basado en Líneas de Producto Software

– *Software Product-Line Engineering (SPLE)*

- SPLE consiste en el desarrollo de una línea de producto software para una familia de productos
- Una familia de productos de software es un conjunto de productos de software asociados a un dominio determinado que tienen:
  - Aspectos comunes que son compartidos por todos sus productos:
    - » Un diseño arquitectónico común
    - » Un conjunto componentes reutilizables
    - » Capacidades y servicios comunes
    - » Tecnologías comunes
  - Aspectos variables que establecen diferencias entre los productos

Desarrollo de Software Basado en Líneas de Producto Software

– *Software Product-Line Engineering (SPLE)*

- Definición:

> • *<<A software product line is a set of software-intensive **systems** sharing a common, managed set of features that satisfy the specific needs of a particular market **segment or mission** and that are developed from a common set of **core assets** in a prescribed way.>>*
>
> ***Software Product Lines: Practices and Patterns.***
> *Clements, P. & Northrop, L. , Addison-Wesley, 2002.*

Desarrollo de Software Basado en Líneas de Producto Software

– *Software Product-Line Engineering (SPLE)*

- Inspirado en los procesos de producción de sistemas físicos
- Producción de aviones, vehículos, computadores, aparatos electrónicos, etc.



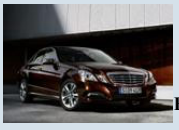
Desarrollo de Software Basado en Líneas de Producto Software

– *Software Product-Line Engineering (SPLE)*

- Aviones:
  - El 60% de las componentes de los Boeing 757 and 767 son comunes




- Coches
  - Probablemente más del 70% de las componentes de los distintos modelos Mercedes Benz E class son comunes.


Berlina

Estate

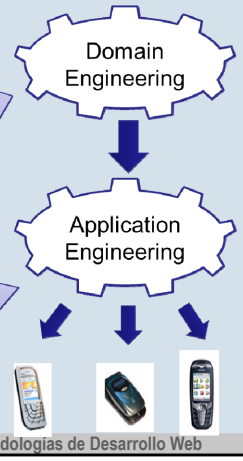
Coupe

Desarrollo de Software Basado en Líneas de Producto Software

– *Software Product-Line Engineering (SPLE)*

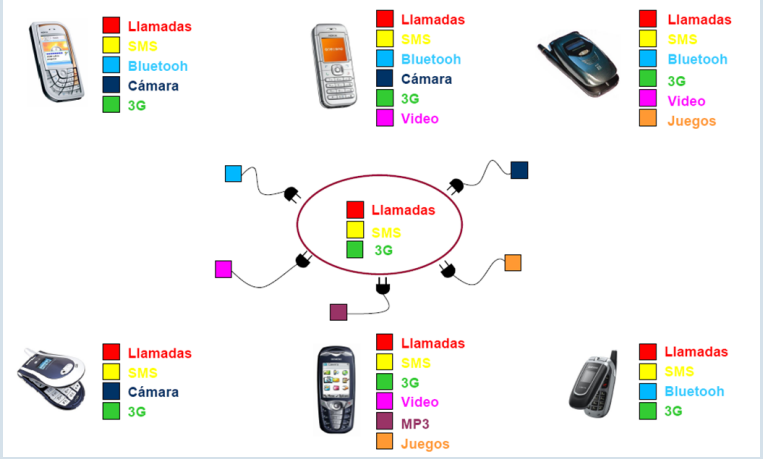
Exploits the commonality and variability across products form the same family by an upfront long-term design of a common set of *core-assets* or *platform*

Core-assets are assembled into customer-specific products by deriving the existing variability across de products



Desarrollo de Software Basado en Líneas de Producto Software

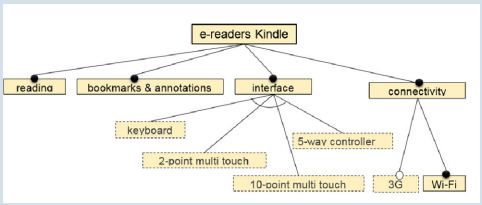
– *Software Product-Line Engineering (SPLE)*

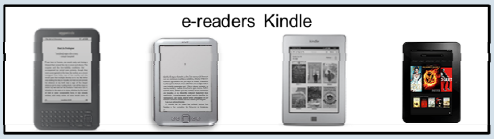


Desarrollo de Software Basado en Líneas de Producto Software

– *Software Product-Line Engineering (SPLE)*

- Se basan en el concepto de característica *(feature)*



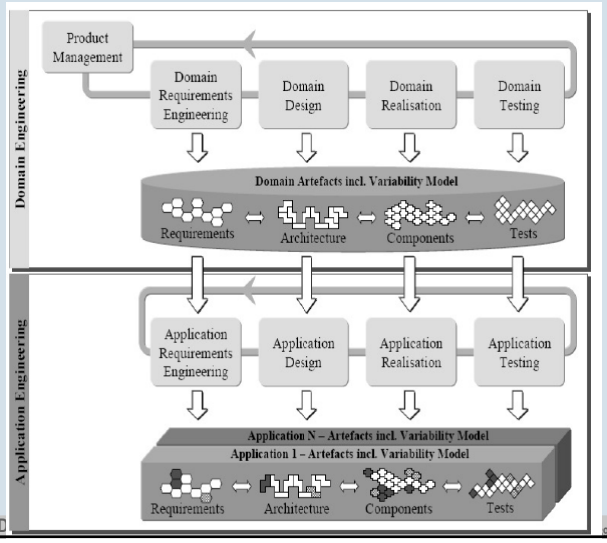




Desarrollo de Software Basado en Líneas de Producto Software

– *Software Product-Line Engineering (SPLE)*

- Enfoque de desarrollo

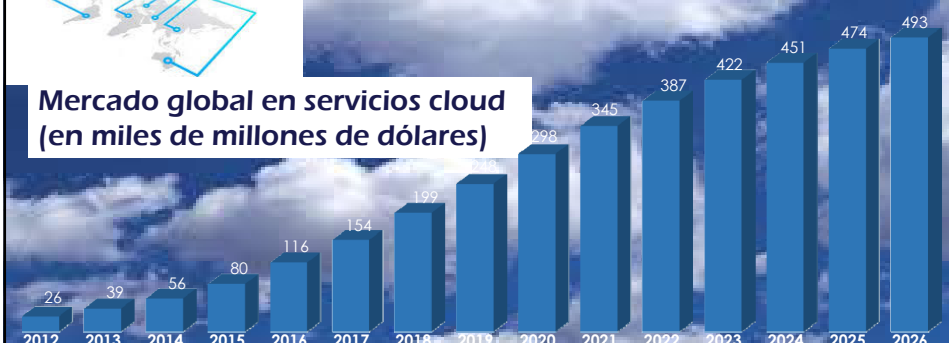


Desarrollo de Software en la Nube

– *Cloud Development*



Mercado global en servicios cloud (en miles de millones de dólares)



Desarrollo de Software en la Nube

– *Cloud Development*

- Ahorro de costes
- Coste basado en uso
- Flexibilidad
- Calidad y fiabilidad
- Seguridad
- Menos mantenimiento

99,95 %

Desarrollo de Software en la Nube

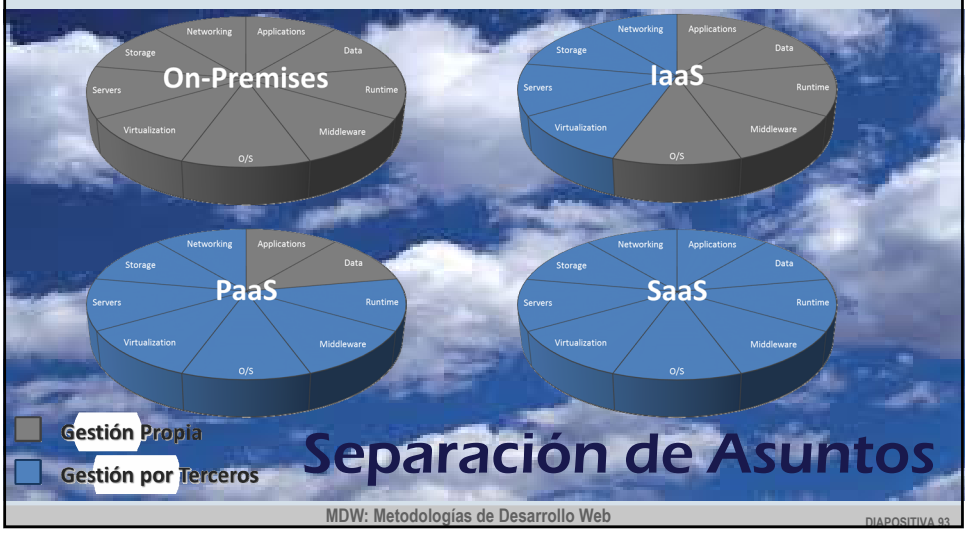
– *Cloud Development*

Windows Azure Platform: Cloud Development Jump Start



Desarrollo de Software en la Nube

– *Cloud Development*



Desarrollo de Software en la Nube

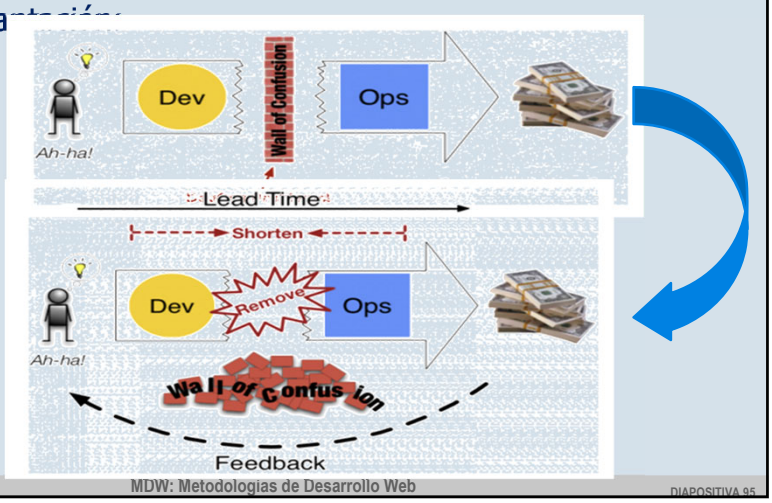
– *Cloud Development*



Desarrollo y Operaciones

– *DevOps*

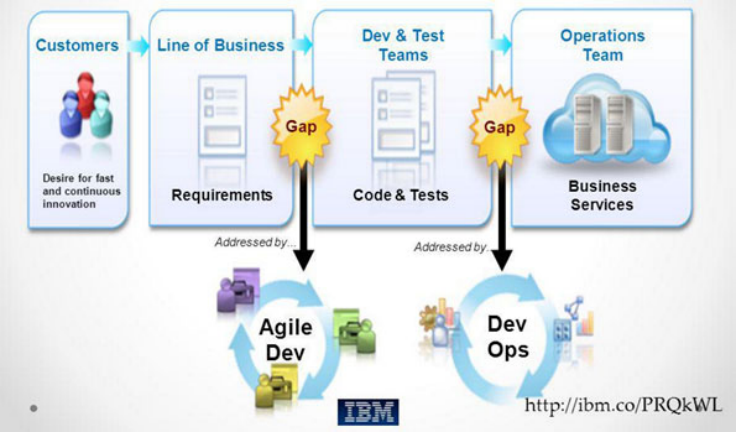
- Tendencia de desarrollo para resolver el problema de la implantación



Desarrollo y Operaciones

– *DevOps*

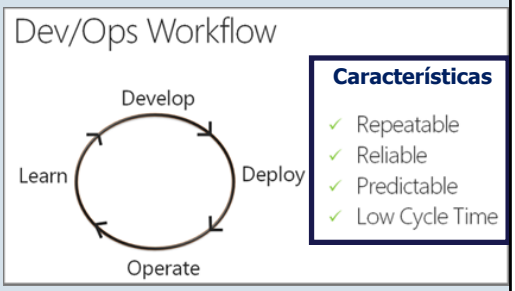
- **Agile + DevOps:**

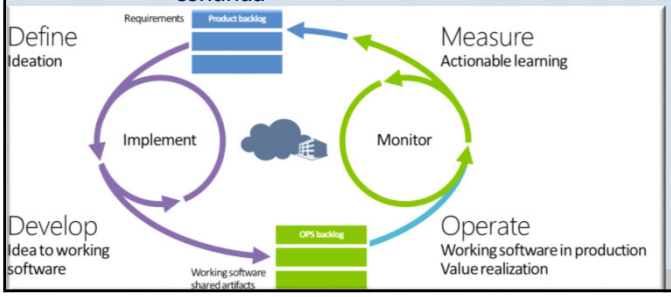


Desarrollo y Operaciones

– *DevOps*

- **Flujo de Trabajo:**
  - **Prácticas**
    - » Automatización Total
    - » Gestión de la Configuración: Control de Versiones
    - » Integración y Entrega continua



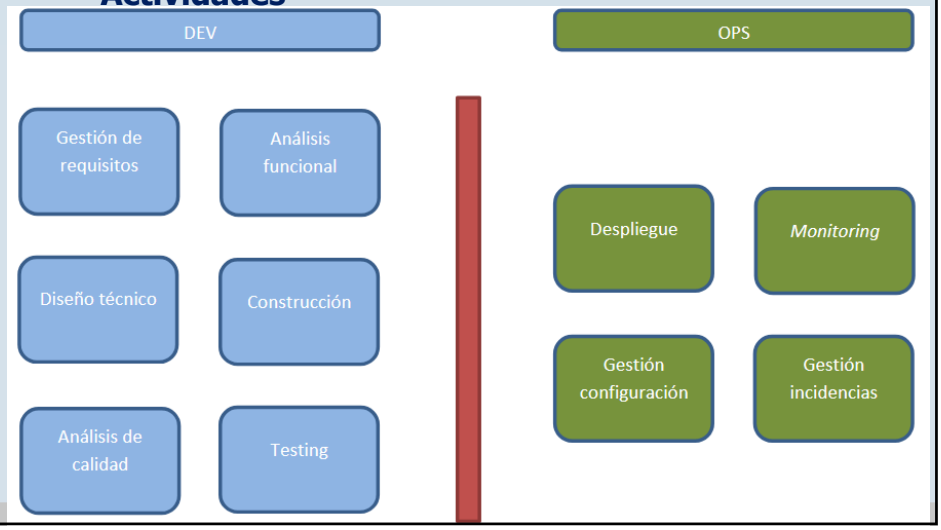


People, Process & Tools – The Essence of DevOps, Richard Campbell

Desarrollo y Operaciones

– *DevOps*

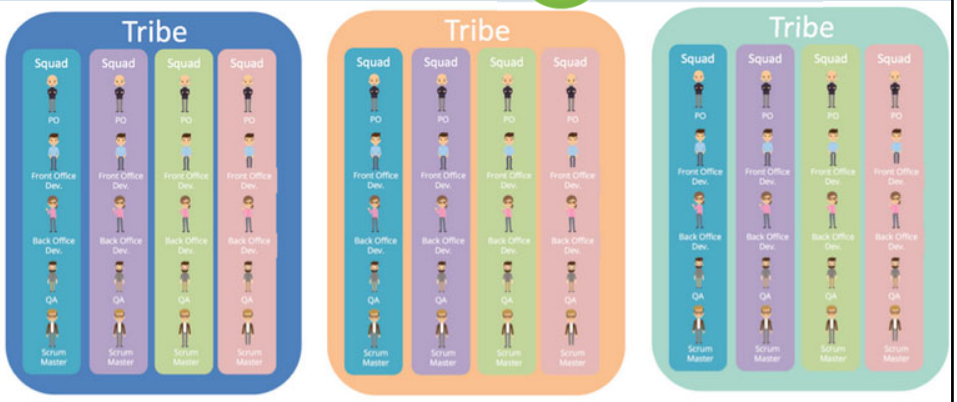
– Actividades



Desarrollo y Operaciones

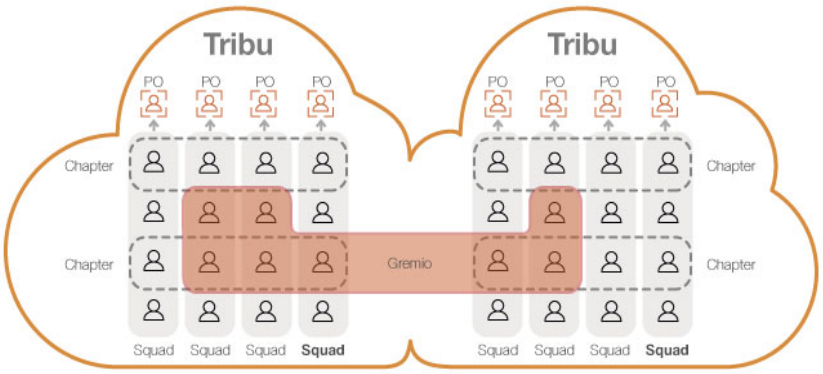
– *DevOps*

- **Organización de Equipo de Desarrollo:**
  - Una de las más adoptadas: Spotify®



Desarrollo y Operaciones – *DevOps*

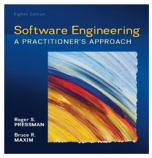
- **Organización de Equipo de Desarrollo:**
  - Una de las más adoptadas: Spotify



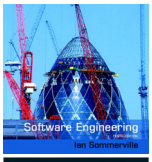
Bibliografía



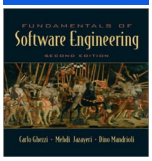
Bibliografía



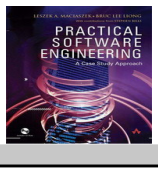
- ***Software Engineering: A Practitioner's Approach 8th Edition***, *Roger Pressman, Bruce Maxim, McGraw Hill Education, © 2014*



- ***Software Engineering, 10th Edition*** Ian Sommerville, University of St Andrews, Scotland, ©2016 http://iansommerville.com/software-engineering-book/



- ***Fundamentals of software engineering 2nd Edition.*** *Carlo Ghezzi, Mehdi Jazayeri, Dino Mandrioli, Prentice Hall 2003: I-XX, 1-604*

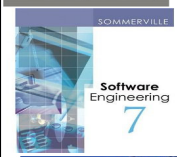


- ***Software Engineering. A Case Study Approach,*** *Maciaszek, L.A. and Liong, B.L, Practical Harlow England, Addison-Wesley, 864p,ISBN 0-321-20465-4, 2005*

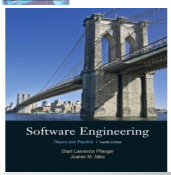
Bibliografía



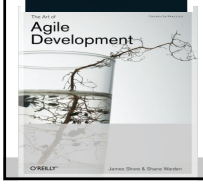
- ***Software Engineering: A Practitioner's Approach 5th Edition***, *Roger Pressman, Bruce Maxim, McGraw Hill Education, © 2005*



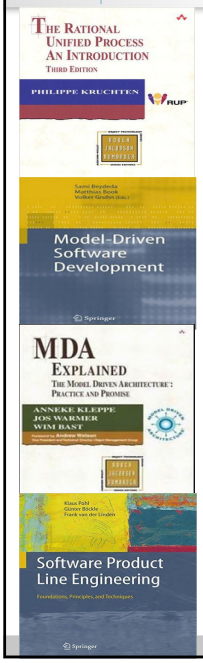
- ***Software Engineering, 7th Edition*** *Ian Sommerville, Addison-Wesley, © 2005*



- ***Software Engineering: Theory and Practice, 4th Edition,*** *Shari Lawrence Pfleeger and Joanne M. Atlee, Prentice Hall, © 2009*



- ***The Art of Agile Development, 1st Edition***, *Shore J., Warden S., O'Reilly Media, © 2007.*



Bibliografía

- ***The Rational Unified Process: An Introduction, 3rd Edition,*** *Philippe Kruchten* Addison Wesley, Object Technology Series, Grady Booch, Ivar Jacobson, and James Rumbaugh, 2003.
- ***Model-Driven Software Development****, Springer, Beydeda, S., Book, M., & Gruhn V., 2005.*
- ***MDA Explained The Model Driven Architecture: Practice and Promise***, Kleppe A., Warmer J., Bast W., Addison Wesley, Object Technology Series, Grady Booch, Ivar Jacobson, and James Rumbaugh, 2004.
- ***Software Product Line Engineering: Foundations, Principles and Techniques.*** *K. Pohl, G. Böckle, and F. Linden, Springer, Germany , © 2005*



Bibliografía

- ***Software Product Lines: Practices and Patterns.*** *Clements, P. & Northrop, L. , Addison-Wesley, 2002.*
- ***Cloud Computing: Concepts, Technology & Architecture****, Ricardo Puttini, Thomas Erl, Zaigham Mahmood, Prentice Hall, 2013.*
- ***Architecting the Cloud: Design Decisions for Cloud Computing Service Models (SaaS, PaaS, and IaaS)****, Michael J. Kavis, John Wiley & Sons, 2014.*
- ***What is DevOps?,*** *Mike Loukides, O'Reilly Media, 2012*

Bibliografía

•***IEEE Standard Glossary of Software Engineering Terminology,*** *IEEE St. 610.121990 1990.*
•*Thomas Kuhn,* ***The Structure of Scientific Revolutions****, 2nd edition, Chicago: University of Chicago Press, 1970*
•*Floyd, R. W ,* ***The paradigms of programming****, Communications of the ACM. 22 (8): 455. doi:10.1145/359138.359140, 1979.*
•Sarat Varanasi, ***COTS Vendor Selection Criteria****, May 13, 2010*
•*Kizcales G., Lamping J., Mendhekar A., Maeda C.,* ***Aspect-Oriented Programming.*** *The 11th European Conference on Object-Oriented Programming (ECOOP), Lecture Notes in Computer Science (LNCS), Springer-Verlag, Vol. 1241, Jyväskylä, Finland, June 9-13, 1997.*
•*Kiczales G., Hilsdale E., Huguin J., Kersten M., Palm J., Griswold W.G.,* ***An Overview of AspectJ.*** *The 15th European Conference on Object-Oriented Programming, Lecture Notes in Computer Science (LNCS), Springer-Verlag, Vol.2072, Budapest, Hungary, June 18-22, 2001.*

Bibliografía

•***IEEE Recommended Practices for Architectural Description of Software-Intensive Systems***. IEEE Std 1471-2000, Software Engineering Standards Committee of the IEEE Computer Society, 21 September 2000.
•Boehm, B. W.“***A Spiral Model of Software Development and Enhancement***”. Computer , 21(5): 61-72 , 1988.
•***ATL/Concepts****, https://wiki.eclipse.org/ATL/Concepts , 2012*
•***Servicios en la Nube con Microsoft Azure: Sistema de Control de Estacionamientos de la Policía Municipal de Móstoles,***
Oscar Moreno Sánchez, Archivo Digital UPM, http://oa.upm.es/42505/, 2016
•***Servicios en la Nube con Microsoft Azure: Desarrollo y Operación de una aplicación Android con DevOps****, Alejandro Mora, Archivo Digital UPM, 2016.*
•***People, Process & Tools – The Essence of DevOps,*** *Richard Campbell, techdays.org/wp-content/uploads/2013/05/Essence-of-DevOps.pptx, 2013.*
•***Use DevOps to Turn IT into a Strategic Weapon,***
http://dev2ops.org/2012/09/use-devops-to-turn-it-into-a-strategic-weapon, 2016


CAMPUS
DE EXCELENCIA
INTERNACIONAL
Universidad Politécnica de Madrid
ETS de Ingeniería de Sistemas
Informáticos
Metodologías de Desarrollo Web
Master en Ingeniería Web
E.T.S. de Ingeniería en Sistemas Informáticos
Universidad Politécnica de Madrid
Curso 2025-2026
Jennifer Pérez y Agustín Yagüe
Universidad Politécnica de Madrid
E.T.S. de Ingeniería en Sistemas Informáticos
Departamento de Sistemas Informáticos